

Table 1: Customers

CustomerID	CustomerName	City
1	John Smith	New York
2	Mary Johnson	Chicago
3	Peter Adams	Los Angeles
4	Nancy Miller	Houston
5	Robert White	Miami

Table 2: Orders

OrderID	CustomerID	OrderDate	Amount
101	1	2024-10-01	250
102	2	2024-10-05	300
103	1	2024-10-07	150
104	3	2024-10-10	450
105	6	2024-10-12	400

Table 3: Payments

PaymentID	CustomerID	PaymentDate	Amount
P001	1	2024-10-02	250
P002	2	2024-10-06	300
P003	3	2024-10-11	450
P004	4	2024-10-15	200

Table 4: Employees

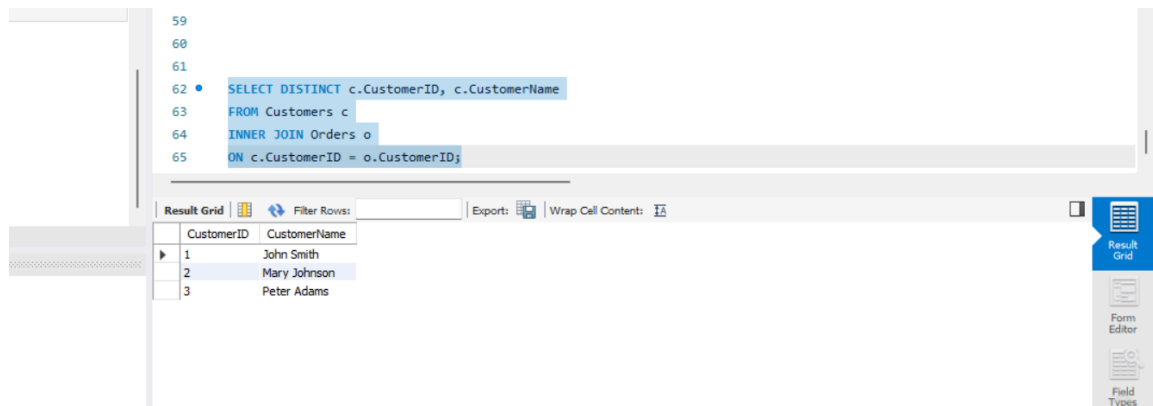
EmployeeID	EmployeeName	ManagerID
1	Alex Green	NULL
2	Brian Lee	1
3	Carol Ray	1
4	David Kim	2
5	Eva Smith	2

1.Retrieve all customers who have placed at least one order.

Answer:

Customers who placed at least one order.

```
SELECT DISTINCT c.CustomerID, c.CustomerName
FROM Customers c
INNER JOIN Orders o
ON c.CustomerID = o.CustomerID;
```



The screenshot shows a SQL query editor with the following query:

```
SELECT DISTINCT c.CustomerID, c.CustomerName
FROM Customers c
INNER JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Below the query, the 'Result Grid' is displayed, showing the results of the query:

CustomerID	CustomerName
1	John Smith
2	Mary Johnson
3	Peter Adams

Here we used **INNER JOIN** because I only want customers who actually have one orders.

2.. Retrieve all customers and their orders, including customers who have not placed any orders

Answer:

All orders with their orders, including not placed orders.

```
SELECT c.CustomerID, c.CustomerName, o.OrderID
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

The screenshot shows MySQL Workbench with a query window containing the following SQL code:

```

62 SELECT DISTINCT c.CustomerID, c.CustomerName
63 FROM Customers c
64 INNER JOIN Orders o
65 ON c.CustomerID = o.CustomerID;
66
67 SELECT c.CustomerID, c.CustomerName, o.OrderID
68 FROM Customers c
69 LEFT JOIN Orders o
70 ON c.CustomerID = o.CustomerID;

```

The Results tab shows the output of the first query (INNER JOIN):

CustomerID	CustomerName	OrderID
1	John Smith	103
1	John Smith	101
2	Mary Johnson	102
3	Peter Adams	104
4	Robert White	105
5	Nancy Miller	106

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
11	11:02:06	CREATE TABLE Payments (PaymentID VARCHAR(10) PRIMARY KEY, CustomerID INT...	0 row(s) affected	0.031 sec
12	11:02:07	INSERT INTO Payments VALUES ('P001', 1, '2024-10-02', 250), ('P002', 2, '2024-10-05', 300),...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.031 sec
13	11:02:37	CREATE TABLE Employees (EmployeeID INT PRIMARY KEY, EmployeeName VARCHAR...	0 row(s) affected	0.047 sec
14	11:02:37	INSERT INTO Employees VALUES ('Alex Green', NULL), ('Brian Lee', 1), ('Carol Ray', ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
15	11:03:09	SELECT DISTINCT c.CustomerID, c.CustomerName FROM Customers c INNER JOIN Orders ...	3 row(s) returned	0.000 sec / 0.000 sec
16	11:10:51	SELECT c.CustomerID, c.CustomerName, o.OrderID FROM Customers c LEFT JOIN Orders o ...	6 row(s) returned	0.000 sec / 0.000 sec

LEFT JOIN helps to show all customers,even if they didn't ordered anything.

3.Retrieve all orders and their corresponding customers, including orders placed by unknown customers.

Answer:

All orders with customers even unknown customers.

```

SELECT o.OrderID, c.CustomerName
FROM Orders o
LEFT JOIN Customers c
ON o.CustomerID = c.CustomerID;

```

The screenshot shows MySQL Workbench with a query window containing the following SQL code:

```

69 LEFT JOIN Orders o
70 ON c.CustomerID = o.CustomerID;
71
72
73 SELECT o.OrderID, c.CustomerName
74 FROM Orders o
75 LEFT JOIN Customers c
76 ON o.CustomerID = c.CustomerID;

```

The Results tab shows the output of the second query (LEFT JOIN):

OrderID	CustomerName
101	John Smith
102	Mary Johnson
103	John Smith
104	Peter Adams
105	

- These shows all orders,even if customers details are missing.

4.. Display all customers and orders, whether matched or not.

Answer:

```
SELECT c.CustomerID, c.CustomerName, o.OrderID
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID
```

UNION

```
SELECT c.CustomerID, c.CustomerName, o.OrderID
FROM Customers c
RIGHT JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

The screenshot shows a SQL IDE interface. On the left is a 'Filter objects' pane with a tree view showing a database schema with tables like 'college', 'company', 'companydb', 'db', 'join_demo', 'sakila', 'sales', 'school', and 'school1'. The 'company' schema is selected. The main area is the 'Query Editor' showing a SQL query:

```
78 SELECT c.CustomerID, c.CustomerName, o.OrderID
79 FROM Customers c
80 LEFT JOIN Orders o
81 ON c.CustomerID = o.CustomerID
82
83 UNION
84
85 SELECT c.CustomerID, c.CustomerName, o.OrderID
86 FROM Customers c
87 RIGHT JOIN Orders o
88 ON c.CustomerID = o.CustomerID;
89
```

Below the query editor is the 'Result Grid' showing the output of the query. It has columns 'CustomerID', 'CustomerName', and 'OrderID'. The results are as follows:

CustomerID	CustomerName	OrderID
1	John Smith	103
1	John Smith	101
2	Mary Johnson	102
3	Peter Adams	104
4	Robert White	NULL
5	Nancy Miller	105
NULL	NULL	105

5.Find customers who have not placed any orders.

Answer:

Customers who didn't place any orders.

```
SELECT c.CustomerID, c.CustomerName
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

The screenshot shows a SQL IDE interface. On the left, a 'SCHEMAS' pane lists various databases including 'college', 'company', 'companydb', 'db', 'join_demo', 'sakila', 'sales', 'school', and 'school1'. The 'company' schema is selected. The main query editor displays the following SQL code:

```

86 FROM Customers c
87 RIGHT JOIN Orders o
88 ON c.CustomerID = o.CustomerID;
89
90
91 • SELECT c.CustomerID, c.CustomerName
92 FROM Customers c
93 LEFT JOIN Orders o
94 ON c.CustomerID = o.CustomerID
95 WHERE o.OrderID IS NULL;
96

```

Below the query editor, the 'Result Grid' shows the following data:

CustomerID	CustomerName
4	Robert White
5	Nancy Miller

- If OrderId is NULL, it means no order was placed.

6.. Retrieve customers who made payments but did not place any orders.

Answer:

Customers who paid but not ordered.

```

SELECT DISTINCT c.CustomerID, c.CustomerName
FROM Customers c
INNER JOIN Payments p
ON c.CustomerID = p.CustomerID
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;

```

The screenshot shows a SQL IDE interface. The main query editor displays the following SQL code:

```

97
98 • SELECT DISTINCT c.CustomerID, c.CustomerName
99 FROM Customers c
100 INNER JOIN Payments p
101 ON c.CustomerID = p.CustomerID
102 LEFT JOIN Orders o
103 ON c.CustomerID = o.CustomerID
104 WHERE o.OrderID IS NULL;

```

Below the query editor, the 'Result Grid' shows the following data:

CustomerID	CustomerName
4	Robert White

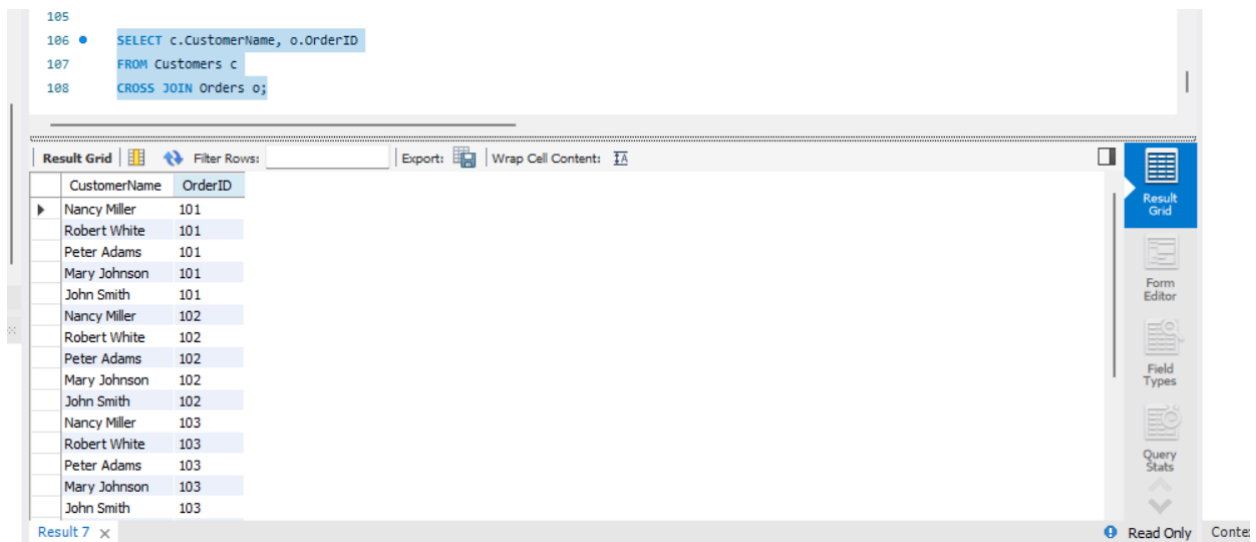
A tooltip message 'Refresh data re-executing the original query' is visible over the result grid.

- First checking payments, then filter those without orders.

7. Generate a list of all possible combinations between Customers and Orders.

Answer:

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
CROSS JOIN Orders o;
```



The screenshot shows a SQL query editor with the following query:

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
CROSS JOIN Orders o;
```

Below the query editor is a 'Result Grid' showing the output of the query. The grid has two columns: 'CustomerName' and 'OrderID'. The results are as follows:

CustomerName	OrderID
Nancy Miller	101
Robert White	101
Peter Adams	101
Mary Johnson	101
John Smith	101
Nancy Miller	102
Robert White	102
Peter Adams	102
Mary Johnson	102
John Smith	102
Nancy Miller	103
Robert White	103
Peter Adams	103
Mary Johnson	103
John Smith	103

- This gives every customer with every order.

8. Show all customers along with order and payment amounts in one table.

Answer:

```
SELECT c.CustomerName, o.Amount AS OrderAmount, p.Amount AS PaymentAmount
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID
LEFT JOIN Payments p
ON c.CustomerID = p.CustomerID;
```

```

109
110 • SELECT c.CustomerName, o.Amount AS OrderAmount, p.Amount AS PaymentAmount
111 FROM Customers c
112 LEFT JOIN Orders o
113 ON c.CustomerID = o.CustomerID
114 LEFT JOIN Payments p
115 ON c.CustomerID = p.CustomerID;

```

Result Grid

	CustomerName	OrderAmount	PaymentAmount
▶	John Smith	150	250
	John Smith	250	250
	Mary Johnson	300	300
	Peter Adams	450	450
	Robert White	200	200
	Nancy Miller	NULL	NULL

Result 8 x

9.. Retrieve all customers who have both placed orders and made payments.

- Combines everything in one table.

Answer:

```

SELECT DISTINCT c.CustomerID, c.CustomerName
FROM Customers c
INNER JOIN Orders o
ON c.CustomerID = o.CustomerID
INNER JOIN Payments p
ON c.CustomerID = p.CustomerID;

```

```

116
117 • SELECT DISTINCT c.CustomerID, c.CustomerName
118 FROM Customers c
119 INNER JOIN Orders o
120 ON c.CustomerID = o.CustomerID
121 INNER JOIN Payments p
122 ON c.CustomerID = p.CustomerID;

```

Result Grid

	CustomerID	CustomerName
▶	1	John Smith
	2	Mary Johnson
	3	Peter Adams